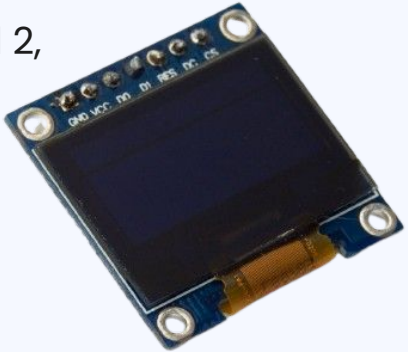
Tasks and Priorities: Intro to RTOS on STM32

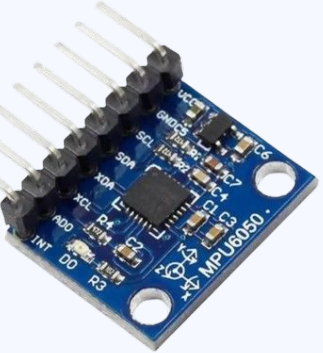
Attendance form

IEEE Penn State - Projects Committee
SP26

Quick Note



- We are assuming you have been with us through workshop 1 and 2, meaning:
  - You have **STM32CubeIDE 1.19.0** installed on your computer
  - You have seen the **I/O Configurator** before
  - You are familiar with setting up **printf()** and serial wire viewer within the CubeIDE
  - Familiarity with your **peripherals** namely the LEDs, OLED screen, and accelerometer in your kit

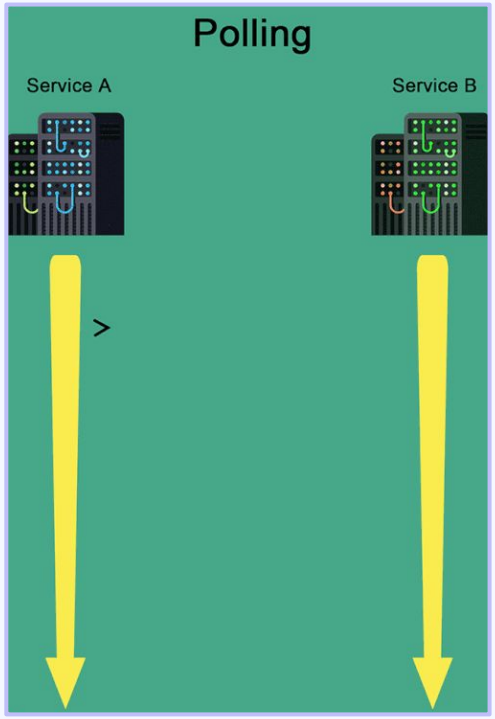


Part 1

Interrupts and Polling

What we've been using - *polling*

- Our code has a main loop `while (1) {...}`
- In loop, we check for certain events/data; button press, accelerometer movement, etc
- This methodology is **polling**
- We write code for CPU to poll hardware in an infinite loop
- Works for simple tasks, like printing IEEE logo on the OLED or Ball Guy™

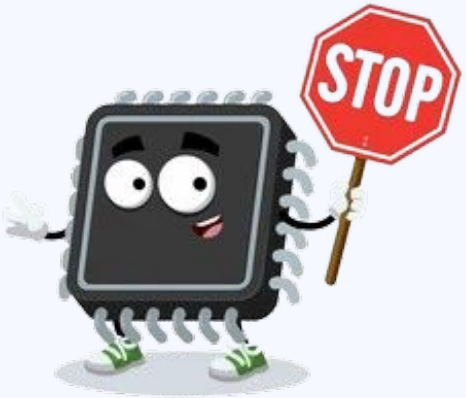


Example - How code executes

```
135  /* USER CODE BEGIN WHILE */
136  while (1)
137  {
138    /* USER CODE END WHILE */
139
140    /* USER CODE BEGIN 3 */
141      uint8_t userButton =  !HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);      //Poll GPIO here to react to user input quicker
142
143      if(userButton) ++minute;       //check button input, increase the minute count if pressed
144      if(second >= 60) {             //increase the minutes and reset the seconds when we reach 60
145         ++minute;
146         second -= 60;
147      }
148      if(minute >= 60) {             //roll-over 60 minutes to an hour
149        ++hour;
150        minute -= 60;
151              }
152      if(hour >= 24) hour -= 24;     //once we hit midnight, reset hours
153
154      ssd1306_fill(&screen, BLACK); //clear the screen
155      //make the string to print to the screen, and ensure that we don't overflow our buffer
156      snprintf(displayTimeBuffer, sizeof(displayTimeBuffer), "%02d:%02d:%02d", hour % 24, minute % 60, second % 60);
157      ssd1306_draw_text(&screen, &font_16x16, displayTimeBuffer, 0, 24);      //print our time, center on the screen
158      ssd1306_update(&screen);  //send the frame to be displayed
159      if(userButton) HAL_Delay(300);    //short delay to make sure m
160  }
161  /* USER CODE END 3 */
162 }
```

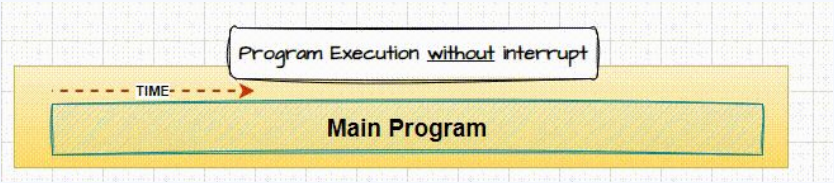
What if it doesn’t work?

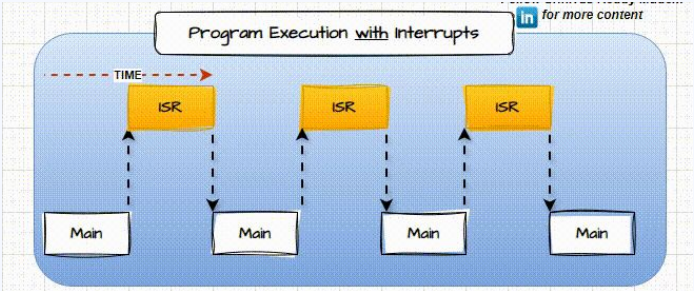
- Polling is great if loop runs very fast compared to the input/output
- But what if:
  - We need intense computation, or to wait for code
  - We don’t want to miss user input
  - We have code that should be prioritized
- **Interrupts** are a broadly used solution to this problem



What are interrupts?

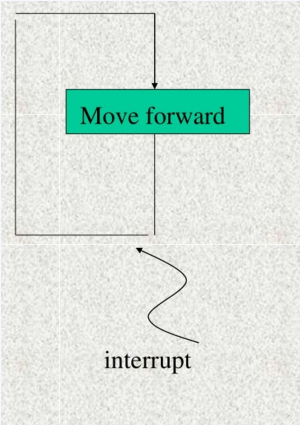
- An **interrupt** is (usually) triggered by external hardware
- Ranges from level change on a GPIO to internal hardware (e.g. timer or communication peripheral)
- It interrupts the CPU's current process, switching it to a block of code called an **Interrupt Service Routine** (ISR)
- CPU will call your function which can decide how to process the input



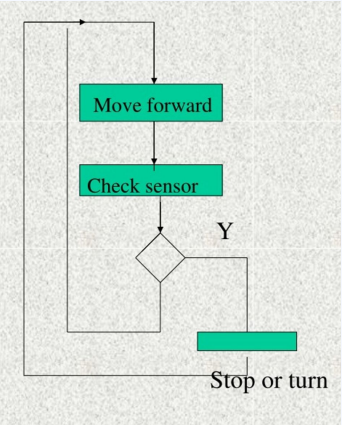


Interrupts vs Polling

- Stops current execution of CPU
- Allows for almost <u>immediate</u> reaction to input → time-sensitive computation
- Requires additional hardware baked into MCU to work



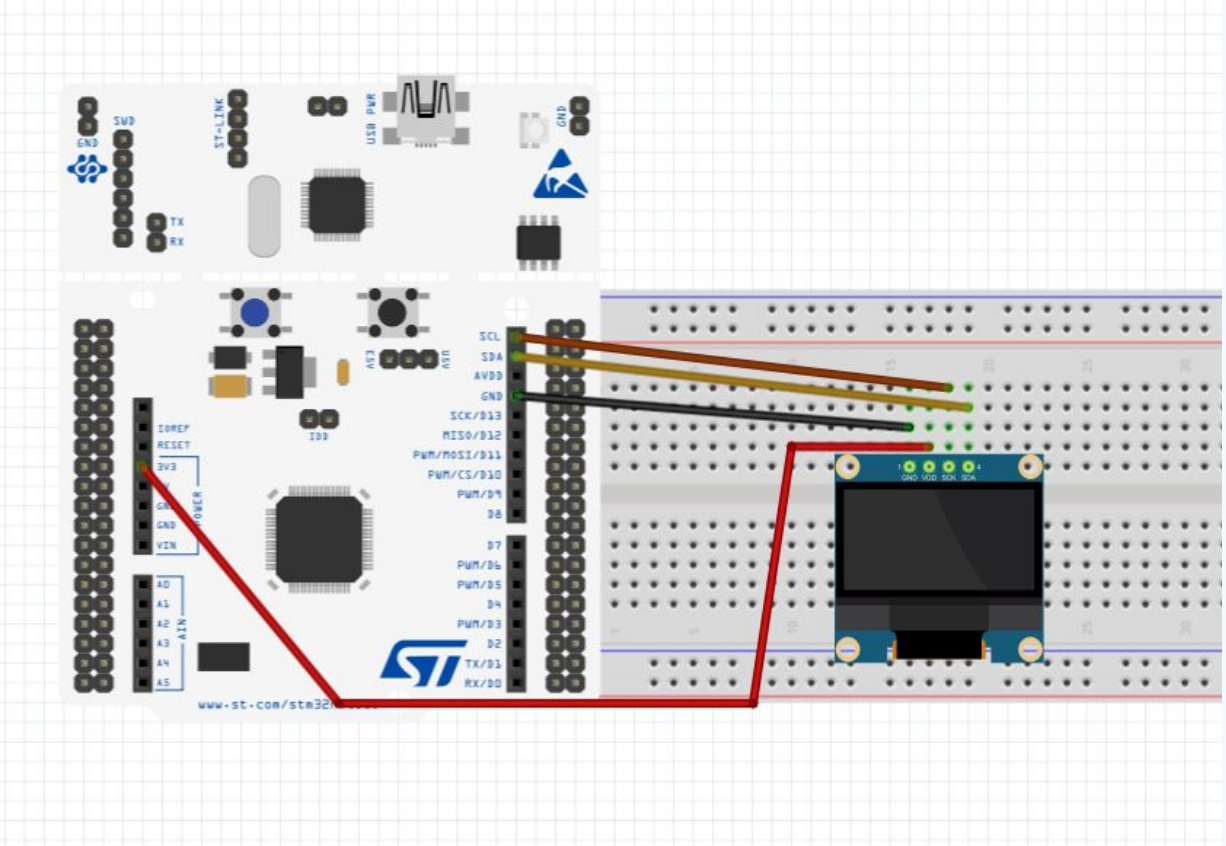
- Must complete all code after polling for input
- “Slow” reaction speed to new inputs
- No special hardware is required



Let's build a clock

- Using the **TIM1** timer peripheral, we will create an interrupt that triggers approximately every 1 second
- We'll use this interrupt to keep time accurately while also update the screen and get user input from the button

Schematic

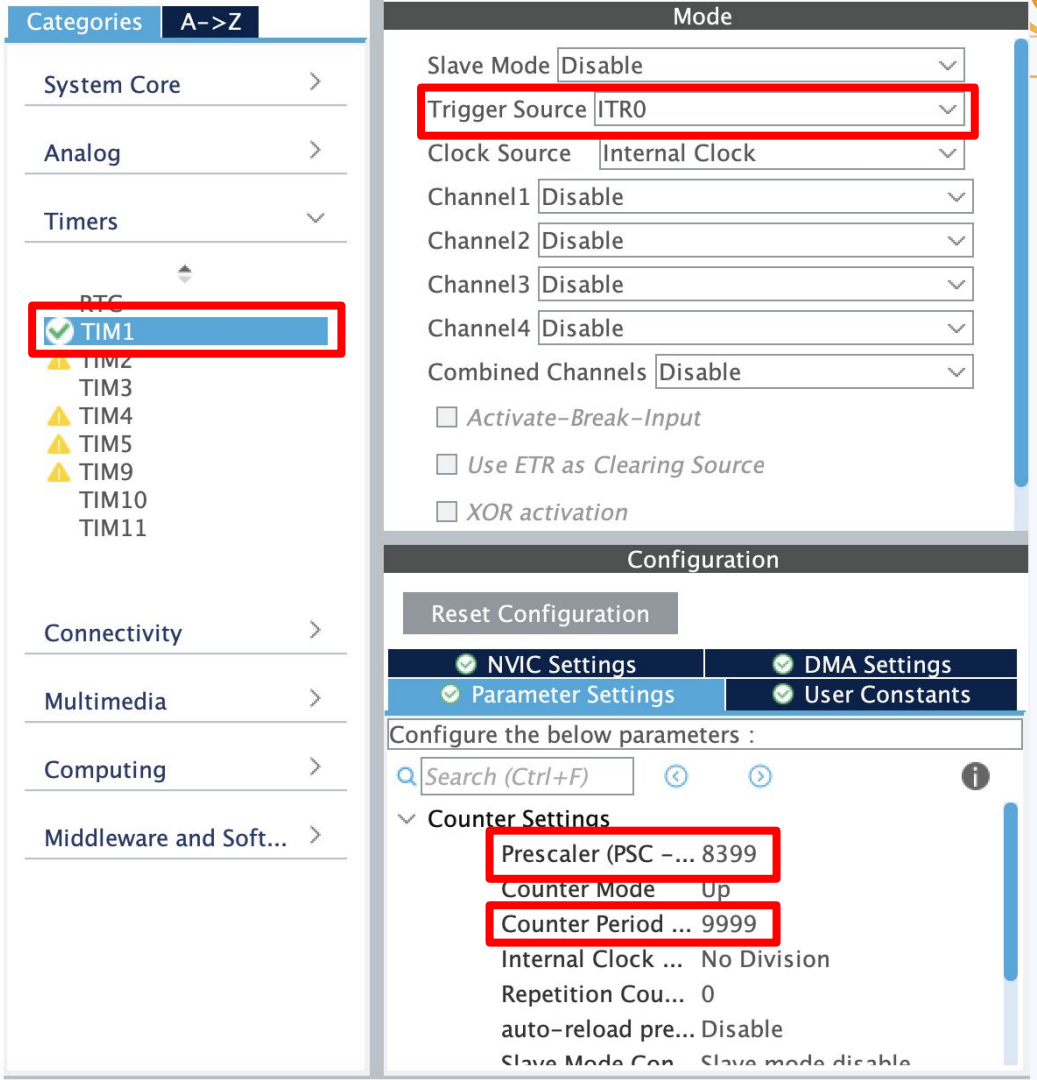

SWD
GND
ST-LINK
USB PWR
GND
TX
RX
SCL
SDA
AVDD
GND
SCK/D13
MISO/D12
PWM/MOSI/D11
PWM/CS/D10
PWM/D9
D8
D7
PWM/D6
PWM/D5
D4
PWM/D3
D2
TX/D1
RX/D0
IOREF
RESET
3V3
GND
VIN
POWER
A0
A1
A2
A3
A4
A5
ANA
IDD
GND VCC SCL SDA

IOC

Interval is given by the equation:

$$\frac{TIM_CLK}{(PSC+1)\times(ARR+1)}$$

PSC:Prescaler
ARR: Counter Period
TIM_CLK: 84MHz
(configurable from clock configurator)



Code

```
/* USER CODE BEGIN PV */
    uint8_t hour;
    uint8_t minute;
    volatile uint8_t second;          //use the

    char displayTimeBuffer[9] = "hello!";
/* USER CODE END PV */
```

```
//---------------Clock ISR-----------------------------
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    ++second;
}

/* USER CODE END 0 */
```

Code Cont.

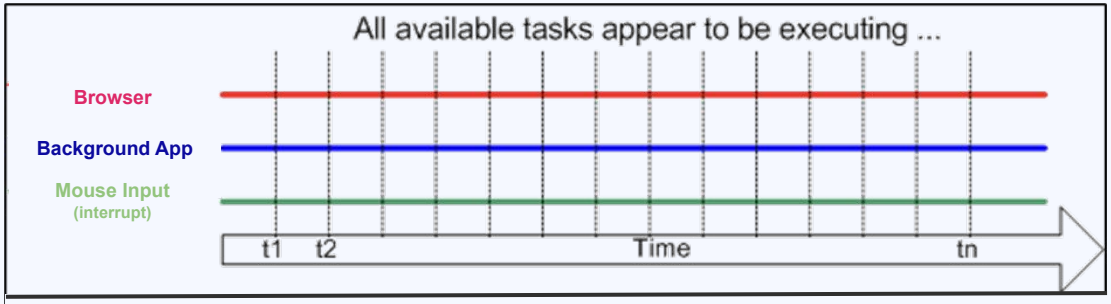
```
/* USER CODE BEGIN 3 */
  uint8_t userButton =  !HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);      //Pol

  if(userButton) ++minute;        //check button input, increase the minute
  if(second >= 60) {              //increase the minutes and reset the seco
        ++minute;
        second -= 60;
    }
    if(minute >= 60) {              //roll-over 60 minutes to an hour
      ++hour;
      minute -= 60;
            }
    if(hour >= 24) hour -= 24;      //once we hit midnight, reset hours

  ssd1306_fill(&screen, BLACK); //clear the screen
```

Part 2

Schedulers

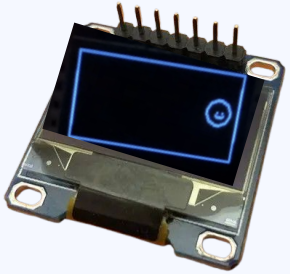
Interrupts and Schedulers

- Operating Systems (OSes) use **schedulers** to switch between different programs.
- When an interrupt is received, the scheduler **finishes** executing the current task and then runs the Interrupt Service Routine.

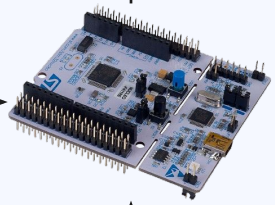

All available tasks appear to be executing ...
Browser
Background App
Mouse Input
(interrupt)
t1
t2
Time
tn

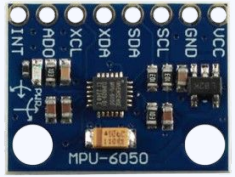
Browser
Background App
Mouse Input
(interrupt)
t1
t2
Time
tn
… but only one task is ever executing at any time.
Browser
Background App
Mouse Input
(interrupt)
t1
t2
Time
tn

2D Ball Guy Bouncer™



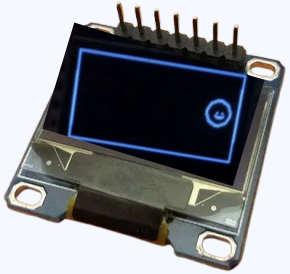
on/off button



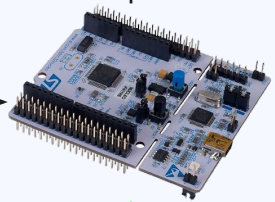


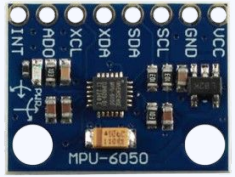
accelerometer/
gyroscope

2D Ball Guy Bouncer™



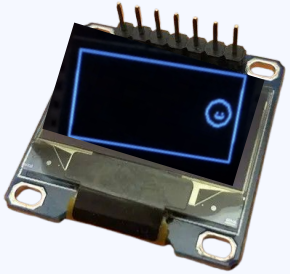
on/off button



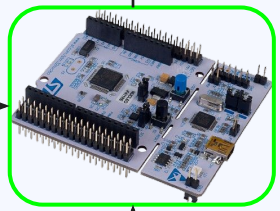


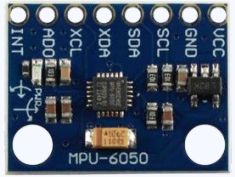
accelerometer/
gyroscope

2D Ball Guy Bouncer™



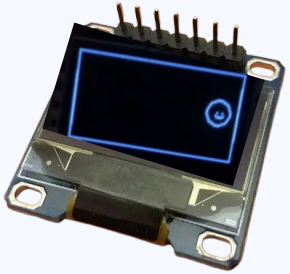
on/off button



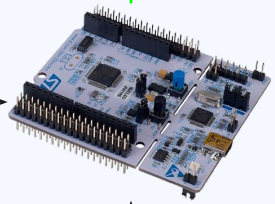


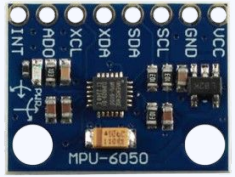
accelerometer/
gyroscope

2D Ball Guy Bouncer™



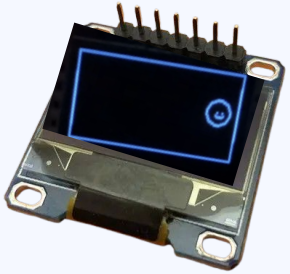
on/off button



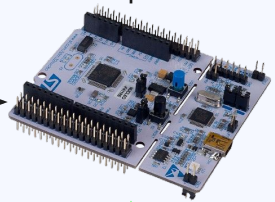


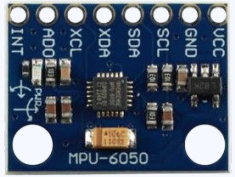
accelerometer/
gyroscope

2D Ball Guy Bouncer™



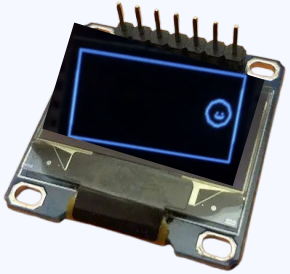
on/off button



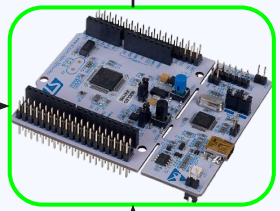


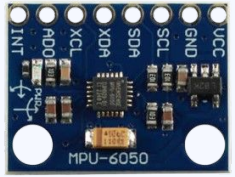
accelerometer/
gyroscope

2D Ball Guy Bouncer™



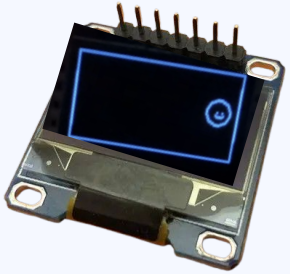
on/off button



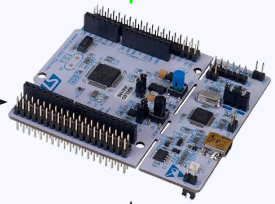


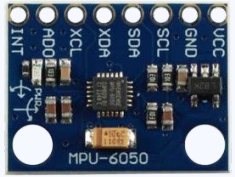
accelerometer/
gyroscope

2D Ball Guy Bouncer™

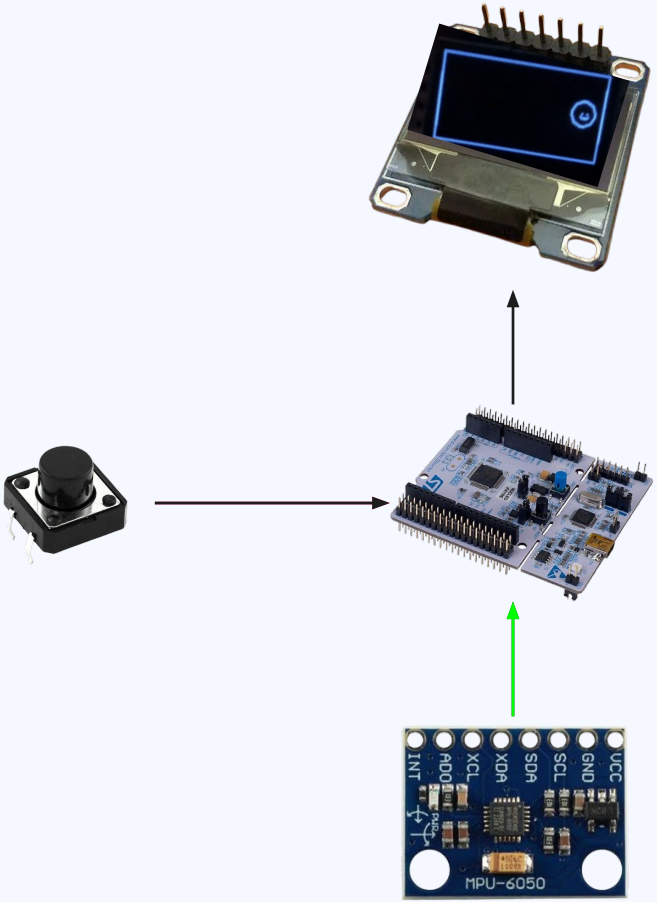


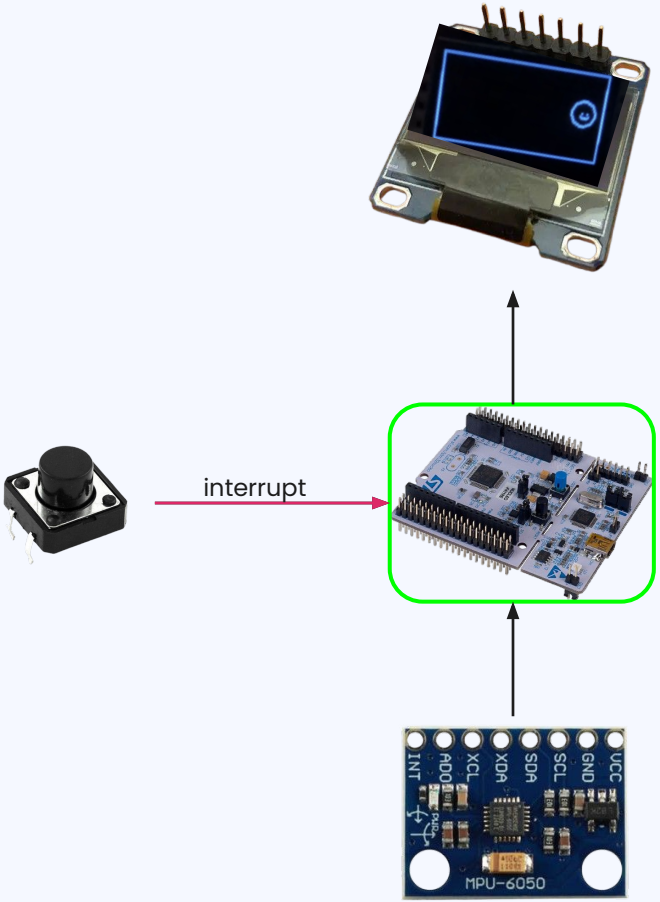
on/off button



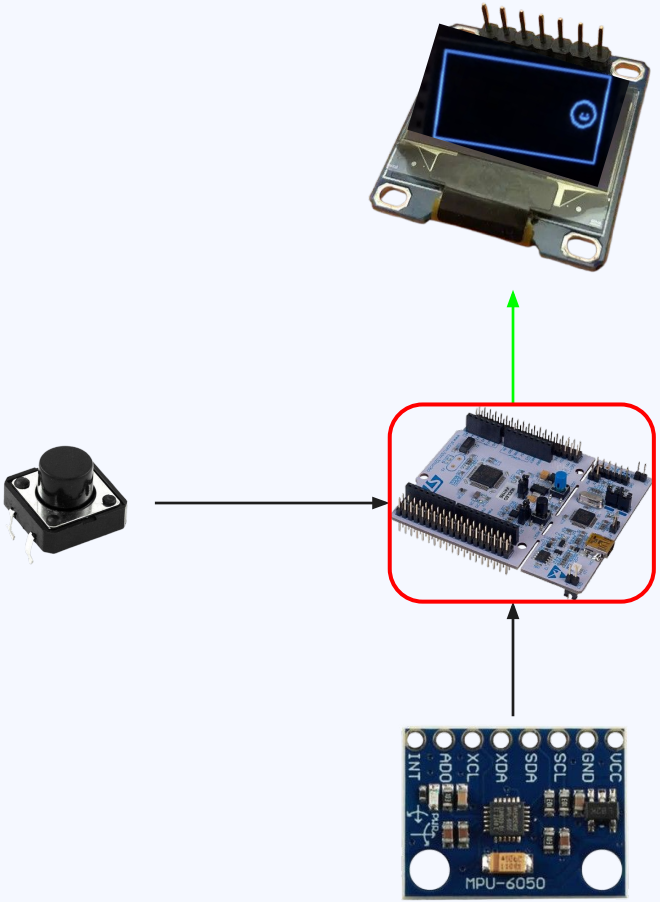


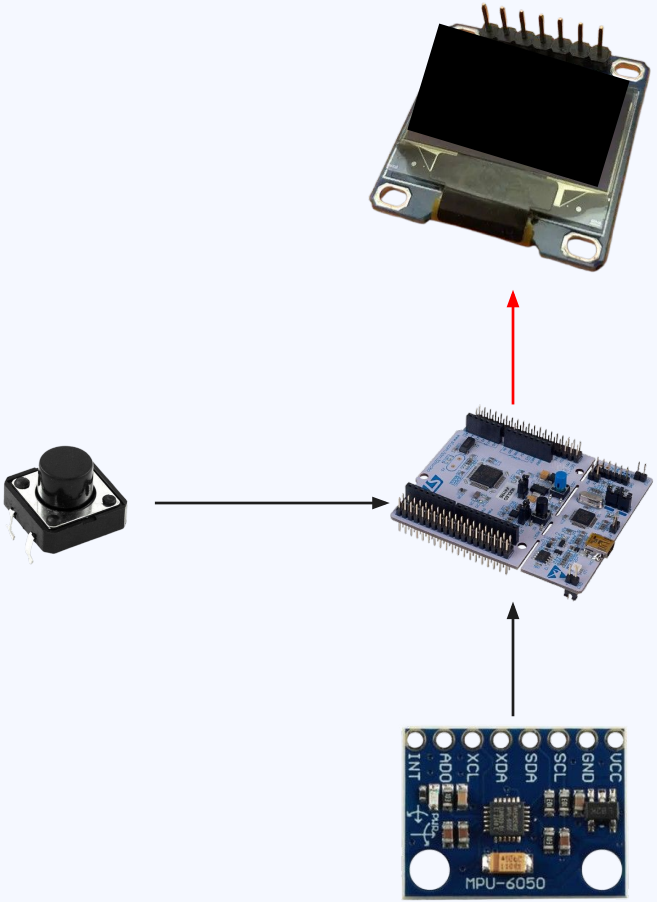
accelerometer/
gyroscope


INT
ADO
XCL
XDA
SDA
SCL
GND
VCC
MPU-6050



Queuing


INT
ADO
XCL
XDA
SDA
SCL
GND
VCC
MPU-6050


INT
ADO
XCL
XDA
SDA
SCL
GND
VCC
MPU-6050

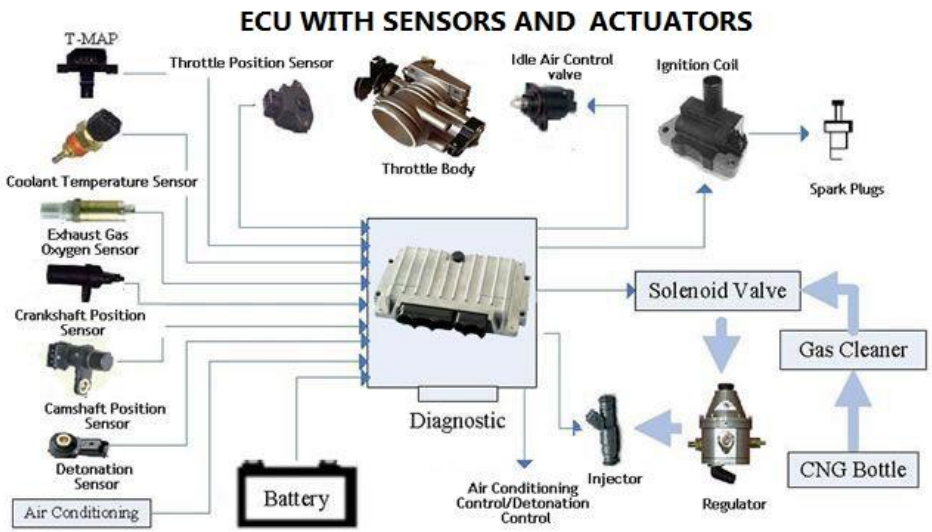
Problems with GPOS Schedulers

Cannot predict when a task will execute

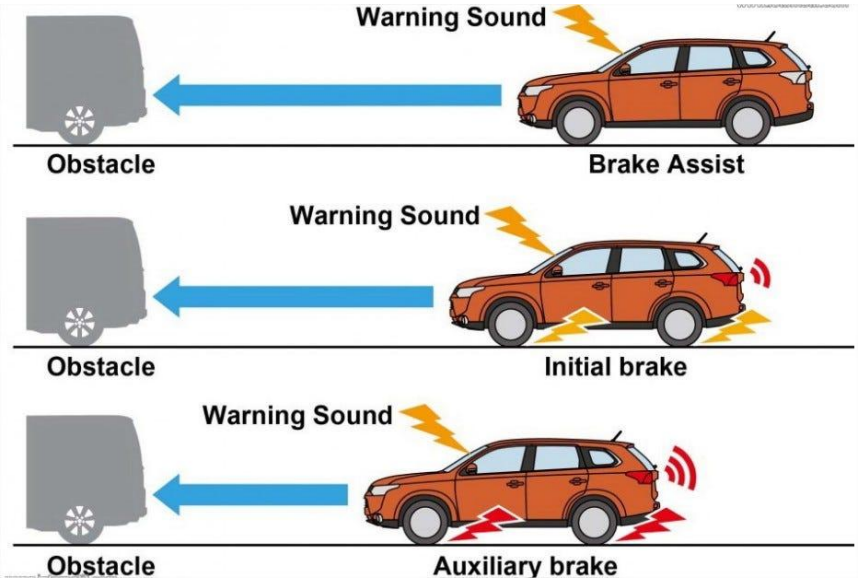
Could execute immediately or only after the current task finishes executing

No guaranteed response times

Not ideal when working in time-sensitive and/or performance-constrained environments

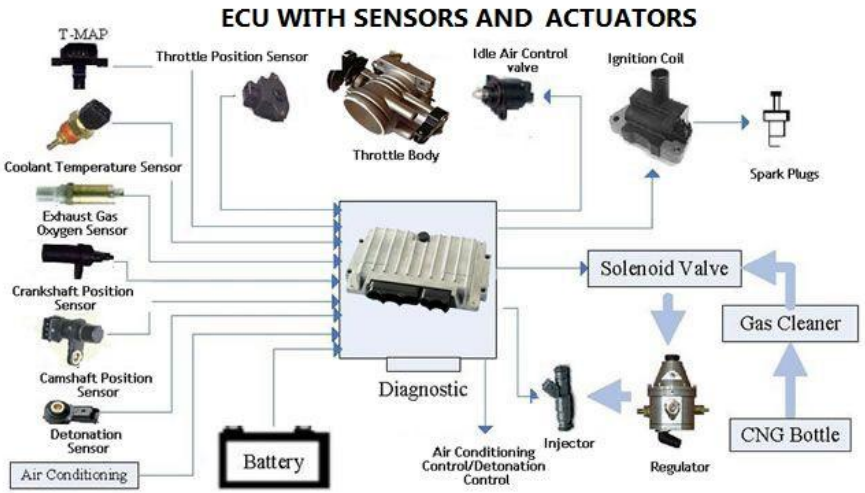


https://robaizkine.com/how-engine-ecu-works-a-comprehensive-guide/

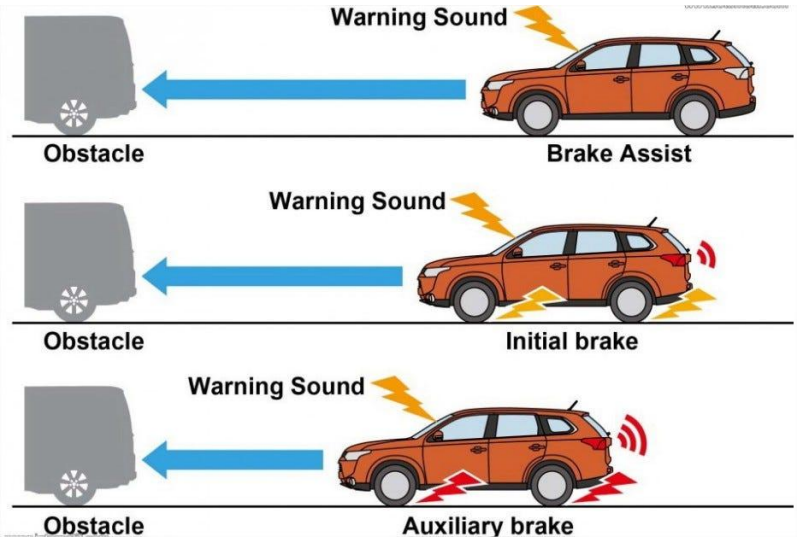


https://blog.roboauto.cz/how-it-works-collision-avoidance-system-d05bd9807f1a?gi=e816b3ed7f6f

Preemption When a less important function is paused during execution to let a more important function run.



https://robaizkine.com/how-engine-ecu-works-a-comprehensive-guide/



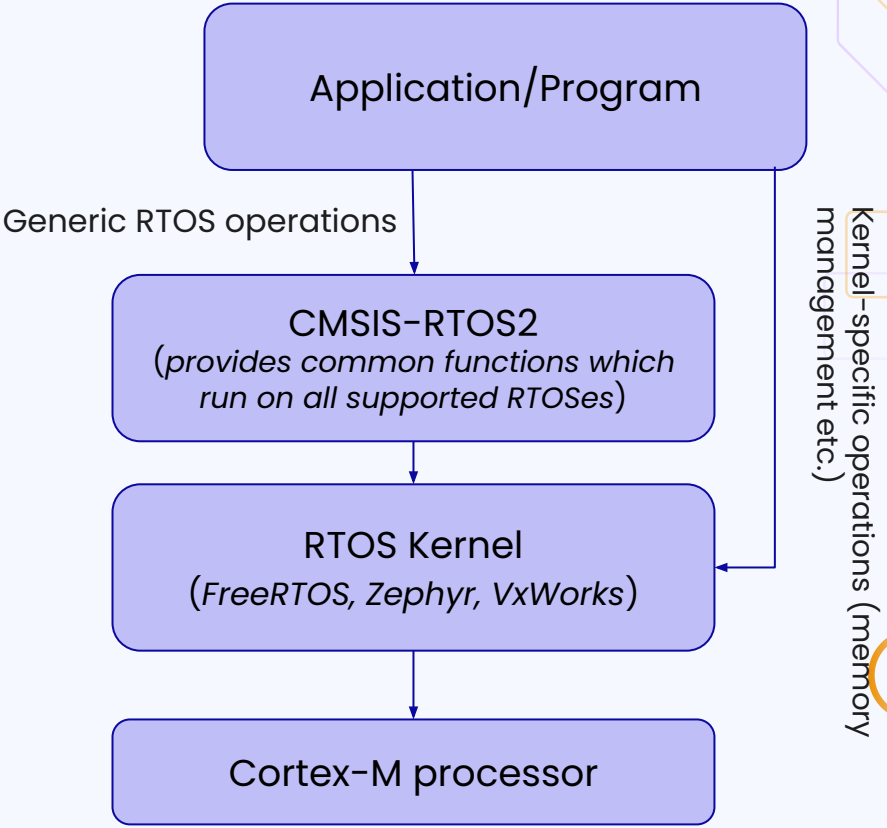
https://blog.roboauto.cz/how-it-works-collision-avoidance-system-d05bd9807f1a?gi=e816b3ed7f6f

RTOS

- RTOS schedulers are **deterministic** because they handle interrupts **preemptively**
- Useful for time-sensitive applications where missing interrupts or not executing tasks completely can result in undesired consequences

CMSIS-RTOS

- Analogous to using C/C++ or Python to build cross-platform applications.



Part 3

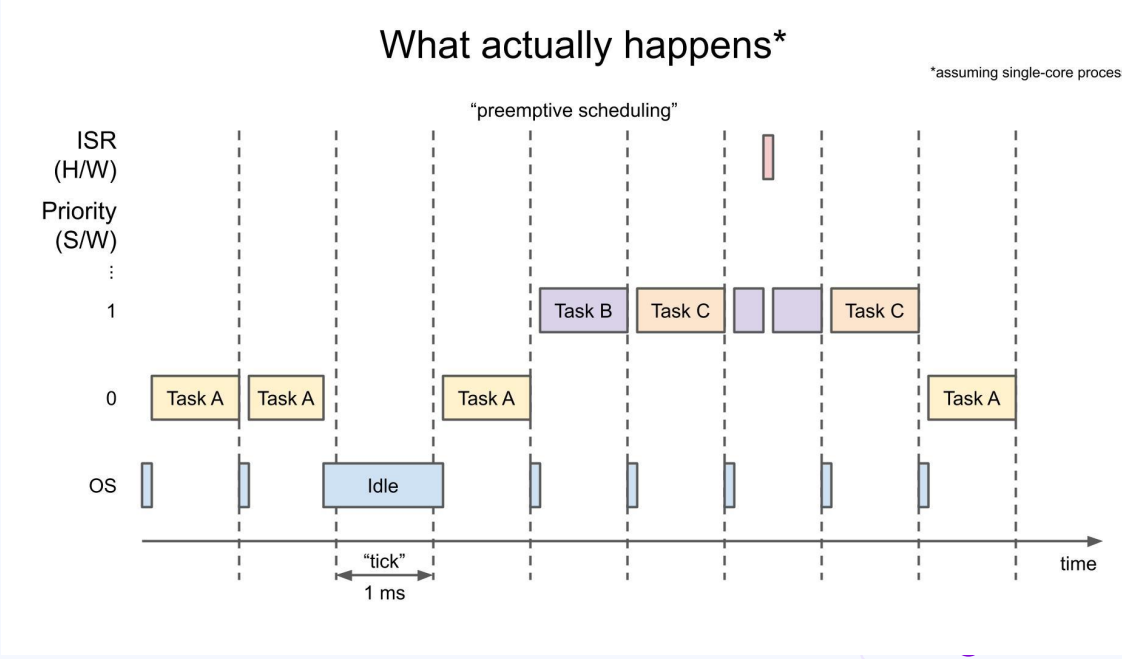
The RTOS Task Priority

In Detail

Tasks in RTOS

Previously, we have been coding all in the main() super loop. But, this can be problematic when we have multiple things to do at precise increments.

Instead with RTOS, we can define multiple tasks that run semi-simultaneously on our STM32



Hands-On Demo 2

Please download and import “IEEErtosDemo_2.zip” from our Github into CubeIDE

What we will be doing:

1) Calculate Pi using the Nilakantha Series

$$\pi = 3 + \sum_{k=1}^{\infty} (-1)^{k+1} \frac{4}{(2k)(2k+1)(2k+2)}$$

2) Display the current value of Pi

3) Blink the LED every 50 ms

All at the same time.

Attempt 1: Non-RTOS Implementation

Here we will do everything in series in the central while loop.

This code is provided for you.

```c
while (1)
{

  // this will run when RTOS is disabled.
  // this infinite loop will operate like other things we have done

  //Calculate next digit of pi
  float term = 4.0f / ((2*k)*(2*k+1)*(2*k+2));
  pi += sign * term;

  sign = -sign;
  k++;

  // Simulate heavier load
  for (volatile int i = 0; i < pow(10,Extra_Computation_Load); i++) {k++;k--;}

  //print pi
  printf("PI = %f\n\r", pi);

  // blink LED
  HAL_GPIO_WritePin(LD2_GPIO_Port,LD2_Pin,GPIO_PIN_SET);
  HAL_Delay(100);
  HAL_GPIO_WritePin(LD2_GPIO_Port,LD2_Pin,GPIO_PIN_RESET);

  // 50 ms delay for LED
  HAL_Delay(50);

  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
```

Make sure your demo variables are as follows:

```c
// demo variables to configure the demonstration
uint8_t RTOS_ENABLE = 0;
uint8_t Extra_Computation_Load = 0; //be careful this has an exponential effect.
```

Lets run!

- Run / Deploy your code file and open the debug terminal on CubeIDE
- You should see a stream of data points in the form “PI = _._____”

- Your LED should also be blinking, but does it every 50 ms?

```
                PI = 3.166667
PI = 3.133333
PI = 3.145238
PI = 3.139683
PI = 3.142713
PI = 3.140881
PI = 3.142072
PI = 3.141255
PI = 3.141840
PI = 3.141407
PI = 3.141736
PI = 3.141480
PI = 3.141683
PI = 3.141519
PI = 3.141654
```

Now lets “simulate” a higher load.

```
uint8_t Extra_Computation_Load = 2; //be careful this has an exponential effect.
```

Set the predefined variable “Extra_Computation_Load” to 2 (should of been 0).

Now redeploy the code and observe the result. Do you notice anything different?

If not, increase the variable again and see if anything changes. Don’t go above 4 otherwise it will be very slow.

Attempt 2: Lets use RTOS

Now we will enable RTOS for this demo.

Set the RTOS_ENABLE to 1. This will enable RTOS for this file.

```
uint8_t RTOS_ENABLE = 1;
```

Task 1: Evaluate the Nilakantha Series

```c
void s_calculatePi(void *argument)
{
  /* USER CODE BEGIN s_calculatePi */
  /* Infinite loop */
    int k = 1;
    float sign = 1.0f;

    while (1)
    {
        float term = 4.0f / ((2.0*k)*(2.0*k+1.0)*(2.0*k+2.0));
        pi += sign * term;

        sign = -sign;
        k++;

        // Optional: simulate heavier load
        for (volatile int i = 0; i < pow(10,Extra_Computation_Load); i++) {
            k++;
            k--;
        }
        osDelay(10);
    }
  /* USER CODE END s_calculatePi */
}
```

This code has been completed for you.

This task has an infinite loop, much like the series implementation, only it is its own task.

Task 2: Blink the LED

```c
void s_blinkLED(void *argument)
{
  /* USER CODE BEGIN s_blinkLED */
  /* Infinite loop */
  for(;;)
  {
    // blink LED
    HAL_GPIO_WritePin(LD2_GPIO_Port,LD2_Pin,GPIO_PIN_SET);
    osDelay(100);
    HAL_GPIO_WritePin(LD2_GPIO_Port,LD2_Pin,GPIO_PIN_RESET);

    // 50ms delay
    osDelay(50);
  }
  /* USER CODE END s_blinkLED */
}
```

This code has been completed for you.

This task has an infinite loop, much like the series implementation, only it is its own task.

Task 3: Print pi

```c
void s_monitorStatus(void *argument)
{
  /* USER CODE BEGIN s_monitorStatus */
  /* Infinite loop */
     for(;;)
     {
       printf("PI = %f\n\r", pi);
       osDelay(10);
     }
  /* USER CODE END s_monitorStatus */
}
```

This code has been completed for you.

This task has an infinite loop, much like the series implementation, only it is its own task.

Lets run again!

Deploy your code again and see if anything is different.

You should notice the LED runs much faster and the “PI = _._____” should be printing much more frequently.

Now lets “simulate” a higher load again.

```
uint8_t Extra_Computation_Load = 10; //be careful this has an exponential effect.
```

Set the predefined variable “Extra_Computation_Load” to 10 now.

Now redeploy the code and observe the result. Do you notice anything different with RTOS enabled?